Distributed Computing Systems Assignment

Group Assignment -Maximum 5 students per group (All Group Members Admission Numbers and Names should indicated on the front page)

Class presentation & Submission: 3rd /November/2026

Submission email: ***studentassignment690@gmail.com***

Assignment Title: Building a Simple Distributed Chat Application

Objective:

The aim of this assignment is to help students understand the principles of distributed computing by creating a simple chat application that operates over a network of multiple server nodes and clients. Students will learn about client-server communication, message broadcasting, and basic fault tolerance.

Assignment Description:

You are required to develop a simple distributed chat application that allows multiple clients to send and receive messages in real-time. The system should include at least two server nodes that can manage client connections and relay messages between clients.

Methodology to Achieve the Assignment:

Step 1: Design the System Architecture

- **Define Components**: Identify the components of the system, which will include:
  - **Client**: The application that users will interact with.
  - **Server**: The backend that handles communication between clients.
- **Communication Protocol**: Decide on a protocol for communication (e.g., TCP sockets).

Step 2: Set Up the Development Environment

- **Choose a Programming Language**: Select a programming language for implementation (e.g., Python, Java, or Node.js).
- **Install Necessary Libraries**: Ensure you have the necessary libraries for socket programming and any other required dependencies.

Step 3: Implement the Server

- **Create Server Nodes**: Write code to handle multiple server nodes.
  - Each server should be capable of accepting client connections.

    - Implement functionality to broadcast messages received from one client to all other connected clients.
- **Manage Client Connections**: Use threading or asynchronous programming to manage multiple clients simultaneously.

Step 4: Implement the Client

- **Create the Client Interface**: Develop a simple command-line or graphical user interface (GUI) for the client.
- **Connect to Server**: Implement functionality for the client to connect to the server and send messages.
- **Receive Messages**: Ensure the client can listen for and display messages from other clients.

Step 5: Implement Basic Fault Tolerance

- **Server Failover**: Implement a basic mechanism to handle server failures. For example:
    - If one server goes down, clients should be able to reconnect to another server.
- **Message Persistence**: Optionally, you can implement a simple log to persist messages in case of server crashes.

Step 6: Testing the System

- **Test Functionality**: Ensure all functionalities work as expected:
    - Multiple clients can connect.
    - Messages are broadcasted correctly.
    - System can handle server failures.
- **Simulate Load**: Test how the system performs under load by connecting multiple clients and sending messages simultaneously.

Step 7: Documentation

- **Code Documentation**: Ensure that the code is well-commented and easy to understand.
- **User Guide**: Create a README file that includes:
    - Instructions to set up and run the application.
    - Explanation of how to use the chat application.
- **Report**: Write a report (2-3 pages) covering:
    - The architecture of your chat application.
    - Challenges faced during implementation and solutions implemented.
    - Performance evaluation and potential improvements.

Deliverables:

1. **Screenshot the User/Front Interface (UI)** of the App and make it as front page of the presentation document after the group names

2. **Source Code**: Submit the complete source code of your chat application, organized in a clear directory structure.
3. **Documentation**: Include a comprehensive README file and a user guide.
4. **Report**: Submit a brief report discussing the architecture, challenges, and performance evaluation.

Evaluation Criteria:

- **Functionality**: Does the chat application meet all the specified requirements?
- **Code Quality**: Is the code well-structured, readable, and documented?
- **Documentation Quality**: Is the README and report clear and informative?
- **Creativity**: Are there any additional features implemented that enhance the application (e.g., private messaging, user authentication)?

Submission Guidelines:

- Submit your assignment via Submission email: studentassignment690@gmail.com & Submission: 3rd /Nov/2026
- Ensure that all files are properly named and organized.
